Algorithm for file updates in Python

Project description

In this project, I created a Python algorithm that automatically updates an allow list file by removing IP addresses that are no longer authorized. This simulates a real cybersecurity task where access permissions must be maintained securely and efficiently using automation.

Open the file that contains the allow list

```
import_file = "allow_list.txt"
```

I assigned the file name to a variable so Python knows which file to open and modify.

Read the file contents

```
with open(import_file, "r") as file:
    ip_addresses = file.read()
```

I used a `with` statement and the `.read()` method to safely open and read the file's contents into a variable.

Convert the string into a list

```
ip_addresses = ip_addresses.split()
```

The `.split()` method converts the string of IP addresses into a list, allowing me to remove specific entries individually.

Iterate through the remove list

```
for element in ip_addresses:
    if element in remove_list:
        ip_addresses.remove(element)
```

I used a `for` loop to check each IP address against the `remove_list`. If a match was found, that IP was removed from the allow list.

Remove IP addresses that are on the remove list

This step ensures that only authorized IPs remain in the file. It demonstrates how conditional logic and list manipulation can automate access control.

Update the file with the revised list of IP addresses

```
ip_addresses = " ".join(ip_addresses)

with open(import_file, "w") as file:
  file.write(ip_addresses)
```

I converted the list back into a string and rewrote the file using `.write()`. This updated the allow list with only the permitted IPs.

Summary

This project demonstrates how Python can automate file management tasks. By reading, modifying, and rewriting a text file, I created an algorithm that updates an allow list by removing unauthorized IP addresses. The process uses essential Python functions such as `open()`, `.read()`, `.write()`, `.split()`, and `.remove()`, along with loops and conditionals to handle data efficiently. This approach improves accuracy and saves time when managing access control lists in cybersecurity workflows.